Java Backend Interview Guide

Core Java · OOP & Design Patterns · Java 8 · Coding · Spring Boot · Microservices · CI/CD & Cloud

A dense, explanation-first study PDF. Each topic covers the concept, internals, why interviewers ask it, common mistakes, and small code sketches you can speak through.

How to use this: read a section out loud, then close the PDF and explain the same idea in 90 seconds. If you cannot name a failure mode, you do not own the topic yet.

Contents

1. Core Java — Collections, Exceptions, Multithreading, JVM/JDK/JRE, Memory, String Pool, equals() & hashCode()
2. OOP + Design Patterns — Principles, SOLID, Singleton, Factory, Builder, Strategy
3. Java 8 — Lambdas, Functional Interfaces, Streams, Optional, Method References, map vs flatMap
4. Coding Questions — Strings, Arrays, Collections, HashMap, Streams, Sorting, Frequency Counting
5. Spring Boot — DI, Bean Lifecycle, REST, JPA, Exception Handling, Security, JWT/OAuth, Redis
6. Microservices — API Gateway, Discovery, Communication, Circuit Breaker, Tracing, Load Balancing
7. CI/CD + Cloud — Git, Maven, Jenkins/GitLab CI, Docker, Deployment, AWS Basics

1. Core Java

Core Java interviews at 3–8 years of experience rarely stop at definitions. They test whether you can predict performance, thread-safety, and failure modes. This section is written so you can answer **what it is**, **how the JVM implements it**, and **what breaks in production**.

1.1 Collections Framework

The Collections Framework is a set of interfaces and implementations in `java.util` for grouping objects. The root ideas are **Collection** (a group of elements) and **Map** (key to value). Collection splits into **List** (ordered, duplicates allowed), **Set** (no duplicates), and **Queue/Deque** (processing order).

List

- **ArrayList**: resizable array. Amortized O(1) add at end, O(1) random access, O(n) insert/remove in the middle because elements shift. Default capacity 10; grows by ~1.5x. Use when reads and appends dominate.
- **LinkedList**: doubly linked nodes. O(1) add/remove if you already have the node, O(n) index access. Extra memory per node (prev/next). Rarely the right default in modern Java; ArrayList usually wins even for inserts unless you constantly add/remove at both ends (Deque).
- **Vector**: like ArrayList but every method is synchronized. Global lock, leftover from Java 1.0. Prefer ArrayList + explicit concurrency, or CopyOnWriteArrayList / Concurrent collections.
- **CopyOnWriteArrayList**: every mutation copies the array. Iterators never throw ConcurrentModificationException and see a snapshot. Excellent for rare writes, many reads (listener lists). Terrible if you mutate often.

Set

- **HashSet**: backed by HashMap (dummy values). Unordered, O(1) average add/contains. Allows one null.
- **LinkedHashSet**: insertion-order iteration via a linked list through entries.
- **TreeSet**: NavigableSet on a red-black tree. O(log n), sorted by Comparable or Comparator. No null if natural order is used (null has no compareTo).
- **EnumSet**: bit-vector of enum constants. Extremely fast and compact. Prefer it over HashSet for enums.

Map

- **HashMap**: array of bins. Index = (capacity - 1) & hash. Collisions: linked list, then red-black tree when a bin has 8 nodes **and** table capacity is at least 64. Untreeify at 6. Load factor 0.75. Allows one null key, many null values. **Not thread-safe**.
- **LinkedHashMap**: insertion order or access order (useful as a simple LRU if you override removeEldestEntry).
- **TreeMap**: sorted keys, red-black tree, O(log n).
- **Hashtable**: fully synchronized, no null keys/values. Legacy. Do not recommend it.
- **ConcurrentHashMap (Java 8+)**: no segment locks. Empty bin: CAS. Occupied bin: synchronized on the first node. Reads are lock-free (volatile node fields). Null keys/values forbidden. size() is a striped estimate.
- **IdentityHashMap**: == instead of equals. WeakHashMap: keys are weak references (GC can drop entries).

HashMap put path in words: compute spread hash, find bin, if empty CAS/place node; if collision, walk the chain (or tree), replace equal key or append; if size exceeds threshold, resize (double capacity) and split bins into lo/hi using one extra hash bit. Java 8 resize does **not** rehash every key from scratch the way people still describe from Java 7.

Interview trap: Java 7 HashMap could infinite-loop during concurrent resize. That is a history question. The right answer today is: HashMap is still not thread-safe; use ConcurrentHashMap, and never use Hashtable as the 'modern' fix.

Fail-fast vs fail-safe iterators

ArrayList/HashMap iterators are **fail-fast**: they check `modCount`. A structural change from another thread (or even the same thread except iterator.remove) throws ConcurrentModificationException. This is a **best-effort** check, not a hard guarantee. ConcurrentHashMap and CopyOnWriteArrayList iterators are weakly consistent / snapshot-style: they do not throw CME, and they may not reflect later mutations.

When to pick which collection

```
Need index + appends?                  ArrayList
Need uniqueness, no order?             HashSet
Need uniqueness + insertion order?     LinkedHashSet
Need sorted unique keys?               TreeSet / TreeMap
Need thread-safe map?                  ConcurrentHashMap
Need thread-safe queue?                ConcurrentLinkedQueue / ArrayBlockingQueue / LinkedBlockingQueue
Need bounded hand-off between threads? BlockingQueue (put/take)
Need tiny enum flags?                  EnumSet / EnumMap
```

1.2 Exceptions

An exception is an object representing an abnormal condition. The hierarchy is `Throwable` → `Error` (JVM disasters: OutOfMemoryError, StackOverflowError — usually do not catch) and `Exception`. `RuntimeException` and its children are **unchecked**. Everything else under Exception is **checked** and must be caught or declared.

- **Checked**: IOException, SQLException. The compiler forces you to handle them. Use when the caller **can reasonably recover** (retry file, ask user).
- **Unchecked**: NullPointerException, IllegalArgumentException, IllegalStateException. Programming bugs or contract violations. Do not swallow them.
- **Error**: do not write `catch (Throwable)` in business code. You can hide OOM and leave the process dying slowly.

try / catch / finally / try-with-resources

`finally` always runs if the try block is entered, even after return, **except** if the JVM dies or a thread is stopped in ways that skip it. If both catch and finally throw, the finally exception wins (the try exception is suppressed unless you add it). **try-with-resources** (Java 7+) is the correct way to close AutoCloseable resources. Resources close in reverse order. Exceptions during close are **suppressed** and attached to the primary exception via `getSuppressed()`.

```
try (BufferedReader br = Files.newBufferedReader(path)) {
    return br.readLine();
} catch (IOException e) {
    throw new UncheckedIOException("Failed to read " + path, e);
}
```

Best practices interviewers want

- Throw early, catch late: validate at the boundary; let domain exceptions bubble to a handler.
- Never catch and ignore (`catch (Exception e) {}`). At minimum log with context, or wrap.
- Preserve the cause: `throw new OrderException("pay failed", e)` — never `e.printStackTrace()` as the only handling.
- Do not use exceptions for normal control flow (they are expensive to fill stack traces).
- Custom exceptions: a few meaningful types (NotFound, Conflict, Validation) beat a zoo of empty subclasses.
- In Spring, map exceptions to HTTP at the edge (@ControllerAdvice), not inside every service method.

Interview trap: If asked 'checked vs unchecked', do not stop at compiler rules. Say: checked exceptions leak implementation (IOException) across layers; modern APIs often wrap them in unchecked domain exceptions so service code stays clean.

1.3 Multithreading

A **process** has its own address space. A **thread** shares the heap with other threads in the same JVM but has its own stack, program counter, and local variables. Concurrency bugs come from **shared mutable state** without a happens-before edge.

Creating work

- **Thread** subclass or **Runnable** submitted to an **ExecutorService**. Prefer executors: pooling, lifecycle, and bounded queues.
- **Callable** returns a value and can throw checked exceptions; you get a Future.
- **CompletableFuture** composes async stages (thenApply, thenCompose, thenCombine, allOf, exceptionally).
- Always pass a **named, bounded** thread pool for blocking IO. The common ForkJoinPool is for CPU-style parallel streams, not HTTP calls.

synchronized vs ReentrantLock vs volatile vs atomic

- **synchronized**: intrinsic monitor, auto-release on exit/exception, reentrant, no tryLock, no fairness control. Use as the default mutex.
- **ReentrantLock**: lock() / unlock() in try/finally. Supports tryLock(timeout), lockInterruptibly(), fairness, multiple Condition objects.
- **volatile**: visibility + ordering. A write happens-before a subsequent read of the same variable. **Not atomic** for `count++`. Not a lock.
- **AtomicInteger / LongAdder**: CAS loops. LongAdder is better under heavy contention for stats.
- **ThreadLocal**: per-thread storage (MDC, transaction context). Must remove() on thread-pool threads or you leak memory and leak user data across requests.

Java Memory Model (happens-before)

The compiler, CPU, and JVM may reorder independent operations. Happens-before is the contract that makes writes visible. Important edges: monitor unlock → later lock on the same monitor; volatile write → later volatile read; Thread.start → run(); termination of a thread → Thread.join(); writing a final field in a constructor → after the constructor publishes the object safely.

Remember: Say this sentence: 'volatile gives visibility and ordering, not mutual exclusion or atomicity of compound actions.'

Deadlock, livelock, starvation

Deadlock: two or more threads each hold a lock the other needs. Fix: lock ordering, tryLock with backoff, fewer locks, immutable designs. Livelock: threads keep yielding/retrying and make no progress. Starvation: a thread never acquires a lock (unfair locks + hot threads). Detect deadlocks with `jstack`, VisualVM, or `ThreadMXBean.findDeadlockedThreads()`.

Executors you should name

```
ExecutorService pool = new ThreadPoolExecutor(
    4, 16,                      // core, max
    60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(200),
    new ThreadFactoryBuilder().setNameFormat("http-%d").build(),
    new ThreadPoolExecutor.CallerRunsPolicy()  // backpressure
);
// Never: unbounded LinkedBlockingQueue + large max pool (that is an OOM)
```

Interview trap: parallelStream() uses the common ForkJoinPool. One blocking call inside a parallel stream can stall unrelated parallel work in the whole JVM.

1.4 JVM, JDK, and JRE

- **JDK (Java Development Kit)**: compiler (`javac`), tools (`jcmd`, `jstack`, `javap`), and a JRE. You need this to build.
- **JRE (Java Runtime Environment)**: JVM + standard libraries to **run** bytecode. Modern JDK distributions blurred this; you often ship a custom runtime with `jlink`.
- **JVM (Java Virtual Machine)**: loads class files, verifies bytecode, interprets / JIT-compiles to native code, manages memory and threads. Spec is from Oracle/OpenJDK; implementations include HotSpot, OpenJ9.
- `.java` → `javac` → `.class` (bytecode) → class loader → bytecode verifier → interpreter → C1/C2 JIT (HotSpot) → native code in Code Cache.

Class loading

Delegation model: Bootstrap (rt.jar / modules) → Platform/Extension → Application class loader. A loader asks the parent first (unless a custom loader inverts this). Same class name loaded by two loaders → two distinct classes (ClassCastException surprises in app servers and plugin systems). Java 9 modules (JPMS) add a stronger encapsulation layer on top of this.

1.5 Memory model of a running JVM

- **Thread stacks**: frames, local primitives, object **references**. StackOverflowError if recursion / huge frames. Stack is not where objects live.
- **Heap**: all objects and arrays. Split historically into Young (Eden + Survivor S0/S1) and Old (Tenured). Minor GC collects young; major/full collects old (or more, depending on collector).

- **Metaspace** (Java 8+, native memory): class metadata. Replaced PermGen. Can still OOM if you leak class loaders (redeploying WARs, generating proxies forever).
- **Code cache**: JIT compiled methods.
- **Direct / native memory**: NIO buffers, JNI. Not always visible in heap dumps. `-XX:MaxDirectMemorySize`.

GC algorithms to mention

- **Serial / Parallel**: stop-the-world, Parallel uses multiple GC threads. Throughput oriented.
- **G1 (default in recent JDKs)**: heap in regions, collects garbage-first regions, aiming at a pause-time goal (`MaxGCPauseMillis`).
- **ZGC / Shenandoah**: concurrent, ultra-low pauses, good for large heaps.
- GC is not a memory leak fixer. If you keep a reference (static map, unbounded cache, ThreadLocal on pooled threads), GC cannot help.

Useful tools: `jcmd <pid> VM.native_memory`, heap dump + Eclipse MAT / VisualVM, GC logs (`-Xlog:gc*` on modern JDKs), Micrometer JVM binders in Spring Boot.

1.6 String, immutability, and the String Pool

`String` is a final class. In modern JDKs the payload is a `byte[]` plus a coder flag (compact strings). Immutability means the character data never changes after construction. That enables: safe sharing, use as HashMap keys, caching hash, and the **string intern pool**.

- String literals (`"hello"`) are interned: all copies of the same literal in the same JDK refer to one pooled instance.
- `new String("hello")` creates a **new heap object** whose payload may still point at pooled data; it is **not** `==` to the literal.
- `intern()` returns the pooled equivalent. Over-interning user input can pin memory (the pool lives in heap in modern JDKs, but interned strings live until they are unreachable — still easy to leak if you intern unbounded unique tokens).
- `StringBuilder` / `StringBuffer` for concatenation in loops. `StringBuffer` is synchronized; use StringBuilder unless you have a rare concurrent builder.
- `+` in a loop compiles poorly; in a single expression the compiler typically uses StringBuilder for you.

```
String a = "java";
String b = "java";
String c = new String("java");
String d = c.intern();
System.out.println(a == b); // true  (same pooled literal)
System.out.println(a == c); // false (distinct object)
System.out.println(a == d); // true  (interned)
```

Interview trap: Do not store passwords in String if you can avoid it: you cannot zero the data; it may linger in the pool or heap dumps. char[] can be overwritten.

1.7 equals() and hashCode()

Object identity (`==`) compares references. `equals` compares **logical equality**. Default Object.equals is identity. If you override equals, you **must** override hashCode so equal objects land in the same HashMap/HashSet bucket.

The contract (memorize)

- **Reflexive**: x.equals(x) is true (x != null).
- **Symmetric**: x.equals(y) iff y.equals(x).
- **Transitive**: if x=y and y=z then x=z.
- **Consistent**: no mutation of equality-relevant state while in a hash collection.
- **Non-null**: x.equals(null) is false, never NPE.
- **hashCode**: if equals is true, hashCodes must be equal. Unequal objects may still collide (that is OK, just slower).

What breaks when you get it wrong

- equals without hashCode: HashSet stores 'duplicates'; Map.get returns null for a key that is logically present.
- hashCode without equals: pointless; lookup still uses equals after the bucket is found.

• **Mutable keys**: put in map, mutate a field used in hashCode, object is orphaned in the old bucket — memory leak + lookup miss.

• JPA entities: do not use generated @Id alone if id is null before persist (all new entities look equal). Prefer a business key, or UUID assigned in constructor.

• Asymmetry trap: comparing `getClass()` vs `instanceof` with inheritance (can break symmetry). Records and final classes avoid this mess.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    User user = (User) o;
    return Objects.equals(email, user.email);
}
@Override
public int hashCode() {
    return Objects.hash(email);
}
```

Remember: Prefer Java records or Lombok @EqualsAndHashCode on an explicit immutable field set. Never include lazy collections or mutable timestamps in equality.

2. OOP and Design Patterns

2.1 Four OOP principles

Encapsulation

Bundle state with the operations that own it, and hide representation. In Java: private fields, public methods, packages, modules. Benefit: you can change the array to a map later without breaking callers. Interview depth: encapsulation is also about **invariants** (a BankAccount that never goes negative because withdraw is the only mutator).

Abstraction

Show the essential contract, hide the rest. Abstract classes vs interfaces: abstract class can hold state and protected helpers; interfaces (Java 8+) can have default/static/private methods but still no instance fields (except static constants). Use interfaces for capabilities (`Payable`, `Repository`); abstract classes for shared skeletal implementation (`AbstractList`).

Inheritance

IS-A relationship. Favor **composition over inheritance** when you only want reuse, not polymorphism. Inheritance couples you to the parent's constructor, protected fields, and fragile base class problem. `final` classes (String, many value types) prevent hostile subclassing.

Polymorphism

Compile-time (overloading): same name, different signatures, resolved by the compiler. **Runtime (overriding)**: JVM invokevirtual/invokeinterface using the object's actual type. `@Override` is mandatory in your muscle memory. Covariant return types are allowed; you cannot shrink thrown checked exceptions incompatibly. Static methods are **hidden**, not overridden — a classic trap.

2.2 SOLID

SOLID is five package/class design rules. Interviewers want a one-line definition plus a Java/Spring example.

- **S — Single Responsibility**: one reason to change. An OrderService that also sends email and writes audit SQL will rot. Split: OrderService, EmailPublisher, AuditLogger.
- **O — Open/Closed**: open for extension, closed for modification. New payment types should be new classes (Strategy), not a growing switch in one file.
- **L — Liskov Substitution**: subtypes must honor the parent's contract. A Square extending Rectangle that breaks setWidth/setHeight violates LSP. Prefer separate types.
- **I — Interface Segregation**: many small interfaces beat one fat one. A worker should not be forced to implement `eat()` if it is a robot. In Spring, avoid a 40-method service interface.
- **D — Dependency Inversion**: depend on abstractions. High-level OrderService depends on `PaymentClient`, not `StripeHttpClient`. Spring DI is DIP in practice.

Interview trap: If you only recite the acronym, you fail. Tie Liskov to a test: every test that uses the base type must still pass with the subtype.

2.3 Singleton

Ensure one instance and a global access point. In Spring, a **singleton bean** is usually enough and safer. If you must write one by hand:

- **Eager static field**: simple, thread-safe class-init, no lazy loading.
- **Synchronized getInstance**: correct, slow after startup.
- **Double-checked locking**: must use `volatile` instance; broken without it due to reordering of constructor writes.
- **Initialization-on-demand holder (Bill Pugh)**: inner static class loaded on first use; JVM class-init lock makes it thread-safe and lazy.
- **Enum singleton**: Joshua Bloch's preferred form — serialization-safe, reflection-safe.

```
public enum Clock {
```

```
    INSTANCE;
    public Instant now() { return Instant.now(); }
}

public final class HolderSingleton {
    private HolderSingleton() {}
    private static class Holder {
        static final HolderSingleton I = new HolderSingleton();
    }
    public static HolderSingleton get() { return Holder.I; }
}
```

Interview trap: Serialization can create a second instance unless you implement readResolve. Reflection can invoke a private constructor unless you guard it. Enums sidestep both. Spring singletons are per-container, not per-JVM — two ClassLoaders still mean two instances.

2.4 Factory

A Factory creates objects without exposing `new` to the caller. **Simple factory**: one method with a switch (not a GoF pattern, still useful). **Factory Method**: a class defers instantiation to subclasses (`createParser()`). **Abstract Factory**: a family of related products (WindowsButton + WindowsCheckbox vs Mac variants) without mixing families.

```
public interface Notifier { void send(String to, String body); }

public final class NotifierFactory {
    public static Notifier create(Channel ch) {
        return switch (ch) {
            case EMAIL -> new EmailNotifier();
            case SMS -> new SmsNotifier();
            case PUSH -> new PushNotifier();
        };
    }
}
```

In Spring you often replace handwritten factories with `@ConditionalOnProperty` beans or a Map of strategy beans injected by type. Still mention Factory so you can discuss **decoupling construction from use**.

2.5 Builder

Builder constructs a complex immutable object step by step. It solves telescoping constructors (`User(name)`, `User(name, email)`, ...) and optional fields. Lombok `@Builder` and Java records + a custom builder are common. Validate in `build()` so you never publish a half-valid object.

```
public final class Mail {
    private final String to, subject, body;
    private final List<Path> attachments;
    private Mail(Builder b) { /* assign + copy list */ }
    public static final class Builder {
        private String to, subject, body;
        private final List<Path> attachments = new ArrayList<>();
        public Builder to(String v) { this.to = v; return this; }
        public Builder attach(Path p) { attachments.add(p); return this; }
        public Mail build() {
            Objects.requireNonNull(to);
            return new Mail(this);
        }
    }
}
```

Remember: Builder is about **readable construction of immutables**. Prototype is about cloning. They are not the same pattern.

2.6 Strategy

Strategy encapsulates a family of algorithms and makes them interchangeable at runtime. Classic replacement for giant if/else or switch on type. Composes beautifully with Spring: inject `List<PaymentStrategy>` and pick by `supports(type)`.

```
public interface DiscountStrategy {
```

```
    BigDecimal apply(BigDecimal price);
}
public final class PricingService {
    private final DiscountStrategy strategy;
    public PricingService(DiscountStrategy strategy) { this.strategy = strategy; }
    public BigDecimal quote(BigDecimal price) { return strategy.apply(price); }
}
```

Related patterns (name them if asked to compare): **Template Method** (inherit and override steps; more rigid), **State** (behavior changes with object lifecycle), **Command** (wrap a request as an object for queues/undo).

3. Java 8 Features

Java 8 is still the language baseline in most backend interviews. The theme is **behavior as data**: pass a function where you used to pass an anonymous class.

3.1 Lambdas

A lambda is a concise implementation of a **single abstract method** interface. Syntax: `(args) -> expression` or `(args) -> { statements; }`. Lambdas **capture** effectively final local variables. They are not identical to anonymous classes: `this` inside a lambda is the enclosing class, they do not create a new named type, and they often compile to `invokedynamic` + LambdaMetafactory instead of a synthetic inner class file (implementation detail, but good to mention).

```
list.sort((a, b) -> a.length() - b.length());
Runnable r = () -> log.info("tick");
Predicate<User> active = u -> u.isActive() && u.age() >= 18;
```

3.2 Functional interfaces

A functional interface has exactly one abstract method (SAM). `@FunctionalInterface` makes the compiler enforce that. Default and static methods do not count against the SAM limit.

- **Predicate<T>**: T → boolean. `and`, `or`, `negate`. Filtering.
- **Function<T,R>**: T → R. `compose`, `andThen`. Mapping.
- **Consumer<T>**: T → void. Side effects (logging, forEach).
- **Supplier<T>**: () → T. Lazy create. Used by orElseGet, factory methods.
- **UnaryOperator<T>** / **BinaryOperator<T>**: Function specializations.
- **BiFunction, BiPredicate, BiConsumer**: two arguments.
- Primitive variants (`IntPredicate`, `ToIntFunction`) avoid boxing in hot loops.

```
@FunctionalInterface
interface TriFunction<A,B,C,R> { R apply(A a, B b, C c); }
```

3.3 Method references

Four kinds. They are lambdas with less noise when the method already exists.

- Static: `Integer::parseInt`
- Bound instance: `System.out::println`
- Unbound instance: `String::toUpperCase` (the first arg becomes `this`)
- Constructor: `ArrayList::new` / `User::new`

```
list.stream().map(String::toLowerCase).forEach(log::info);
Map<String, User> users = ids.stream().collect(toMap(id -> id, User::load));
```

3.4 Streams

A Stream is a **pipeline** over data: source → zero or more lazy intermediate ops → one terminal op. Nothing runs until the terminal operation. A stream can be consumed **once**.

- **Sources**: collection.stream(), Stream.of, Files.lines, Stream.iterate, IntStream.range.
- **Intermediate (lazy)**: filter, map, flatMap, distinct, sorted, peek, limit, skip, boxed.
- **Terminal (eager)**: forEach, collect, reduce, count, anyMatch/allMatch/noneMatch, findFirst/findAny, toArray.
- **Short-circuit**: findFirst, anyMatch, limit — they can stop the source early.

Collectors you must know

```
Map<Dept, List<Emp>> byDept = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept));

Map<Dept, Double> avg = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept, Collectors.averagingInt(Emp::salary)));

String csv = names.stream().collect(Collectors.joining(","));

Map<Boolean, List<Emp>> parts = emps.stream()
    .collect(Collectors.partitioningBy(e -> e.salary() > 100_000));
```

Anti-patterns

- Mutating external lists inside forEach — that is a loop in costume.
- Using peek for business logic (it is a debug hook).
- parallelStream on a 20-element list or on blocking IO.
- Collecting to List only to stream again without reason.
- Relying on encounter order with findAny / unordered parallel streams.

3.5 Optional

Optional<T> is a container that may be empty. It exists to make **absence explicit in return types**. It is not a replacement for every null in the system.

- Do: return Optional from find-style methods. Chain map / flatMap / filter / orElseGet / orElseThrow.
- Do not: use Optional as a field, method parameter, or collection element (use empty collection instead).
- Do not: call get() without isPresent — use orElseThrow.
- **orElse(x)** always evaluates x. **orElseGet(supplier)** is lazy. If x is `loadFromDb()`, that difference is a production bug.
- Optional is not Serializable; do not put it in JPA entities.

```
return repo.findByEmail(email)
    .filter(User::isActive)
    .map(User::name)
    .orElseThrow(() -> new NotFoundException(email));
```

3.6 map() vs flatMap()

map transforms each element **one-to-one**: Stream<T> + (T → R) → Stream<R>. If R is itself a Stream or Optional, you get a nested type: Stream<Stream<R>> or Stream<Optional<R>>. **flatMap** transforms **one-to-many** and **flattens**: Stream<T> + (T → Stream<R>) → Stream<R>.

```
// map: List<String> lengths
List<Integer> lens = words.stream().map(String::length).toList();

// flatMap: flatten nested lists
List<String> tags = posts.stream()
    .flatMap(p -> p.tags().stream())
    .distinct()
    .toList();

// Optional: map keeps Optional<Optional<T>>; flatMap flattens
Optional<String> city = userOpt.flatMap(User::address).map(Address::city);

// Stream + Optional
Stream<User> found = ids.stream()
    .map(repo::findById)          // Stream<Optional<User>>
    .flatMap(Optional::stream);   // Stream<User>   (Java 9+)
```

Remember: Interview one-liner: 'map wraps, flatMap unwraps. If your lambda already returns a Stream or Optional, you almost always wanted flatMap.'

4. Coding Questions

Speak while you code: restatement → example → brute force → complexity → optimized approach → tests (empty, one element, duplicates, negatives, unicode). The sketches below are interview-sized, not LeetCode-golf.

4.1 Strings

Reverse a string / check palindrome

```
boolean palindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++; j--;
    }
    return true;
}
// Unicode-aware: use code points, not charAt (surrogate pairs).
```

Anagram

Two strings are anagrams if they have the same character frequencies. Sort both (O(n log n)) or count in an int[256] / HashMap (O(n)).

```
boolean anagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] c = new int[256];
    for (int i = 0; i < a.length(); i++) {
        c[a.charAt(i)]++;
        c[b.charAt(i)]--;
    }
    for (int n : c) if (n != 0) return false;
    return true;
}
```

First non-repeating character

```
Character firstUnique(String s) {
    Map<Character, Integer> freq = new LinkedHashMap<>();
    for (char ch : s.toCharArray()) freq.merge(ch, 1, Integer::sum);
    for (var e : freq.entrySet())
        if (e.getValue() == 1) return e.getKey();
    return null;
}
```

4.2 Arrays

Two Sum

```
int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        Integer j = seen.get(target - nums[i]);
        if (j != null) return new int[]{j, i};
        seen.put(nums[i], i);
    }
    throw new IllegalArgumentException("no pair");
}
```

Kadane — maximum subarray sum

```
int maxSubArray(int[] a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

Rotate array right by k

```
void rotate(int[] a, int k) {
    k %= a.length;
    reverse(a, 0, a.length - 1);
    reverse(a, 0, k - 1);
    reverse(a, k, a.length - 1);
}
```

Move zeros / Dutch national flag

Two pointers: write-index for non-zeros. For 0/1/2 sort (Dutch flag), three pointers low/mid/high in one pass — mention this when they ask about partitioning.

4.3 Collections & HashMap patterns

HashMap is the default frequency and grouping tool. `merge`, `computeIfAbsent`, and `getOrDefault` show fluency.

```
Map<String, Long> freq = new HashMap<>();
for (String w : words) freq.merge(w, 1L, Long::sum);

Map<String, List<Order>> byUser = new HashMap<>();
for (Order o : orders)
    byUser.computeIfAbsent(o.userId(), k -> new ArrayList<>()).add(o);

// LRU cache sketch
class LRU<K,V> extends LinkedHashMap<K,V> {
    private final int cap;
    LRU(int cap) { super(cap, 0.75f, true); this.cap = cap; }
    protected boolean removeEldestEntry(Map.Entry<K,V> e) {
        return size() > cap;
    }
}
```

Interview trap: If they ask HashMap internals during coding, mention treeification and why mutable keys are poison — then get back to the problem. Do not derail a 20-minute coding round.

4.4 Streams-based solutions

```
// top-N words
List<String> top = words.stream()
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(3)
    .map(Map.Entry::getKey)
    .toList();

// flatten + distinct + sort
List<Integer> ids = teams.stream()
    .flatMap(t -> t.memberIds().stream())
    .distinct()
    .sorted()
    .toList();
```

4.5 Sorting

- `Arrays.sort` / `List.sort`: primitives use dual-pivot quicksort; objects use TimSort (stable). Stability matters when you sort by last name then first name in two passes.
- Custom: `people.sort(Comparator.comparing(Person::age).thenComparing(Person::name));`
- nulls: `Comparator.nullsLast(Comparator.naturalOrder())`.
- Know O(n log n) comparison lower bound; counting/radix sort when the key domain is small.
- Interview trap: `Comparable` is the class's natural order; `Comparator` is an external strategy (Strategy pattern).

4.6 Frequency counting playbook

Whenever the problem says 'most frequent', 'anagrams', 'duplicates', 'majority element', start with a count map (or a fixed alphabet array). Then decide: a second pass, a heap for top-K (O(n log k)), or Boyer-Moore if majority is > n/2.

```
int majority(int[] a) { // Boyer-Moore
    int cand = 0, count = 0;
    for (int x : a) {
        if (count == 0) cand = x;
        count += (x == cand) ? 1 : -1;
    }
    return cand; // verify with a second pass if needed
}

List<String> topK(String[] words, int k) {
    Map<String, Long> f = Arrays.stream(words)
        .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
    PriorityQueue<String> pq = new PriorityQueue<>(
        (x, y) -> f.get(x).equals(f.get(y)) ? y.compareTo(x) : Long.compare(f.get(x), f.get(y)));
    f.keySet().forEach(w -> { pq.offer(w); if (pq.size() > k) pq.poll(); });
    List<String> out = new ArrayList<>(pq);
    out.sort((x, y) -> f.get(y).compareTo(f.get(x)));
    return out;
}
```

5. Spring Boot

Spring Boot is opinionated Spring: auto-configuration, starter POMs, an embedded server, and production endpoints. Interviews probe **the container**, not `@SpringBootApplication` as a magic annotation. `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`.

5.1 Dependency Injection

The IoC container owns object graphs. You declare beans; Spring wires them. DI styles:

- **Constructor injection (preferred)**: required deps as final fields, immutable, easy tests, circular deps fail at startup.
- **Setter injection**: optional deps, or reconfiguration. Avoid for required collaborators.
- **Field injection (@Autowired on field)**: short, bad for tests and hidden deps. Many teams ban it.

`@Component` / `@Service` / `@Repository` / `@Controller` are stereotype annotations on classes. `@Bean` methods on `@Configuration` classes are for wrapping third-party types or when you need the method body. `@Configuration` is CGLIB-proxied so `@Bean` calls go through the container (singleton guarantees). `@Configuration(proxyBeanMethods = false)` is faster and means a direct Java call creates a new instance.

Interview trap: Circular dependency: Boot 2.6+ forbids it by default. Fix the design (extract a third type), or last-resort `@Lazy`. Do not brag about enabling circular references.

5.2 Bean lifecycle

Order you should be able to recite:

- Load bean definitions (from scan, @Bean, XML, auto-config).
- **BeanFactoryPostProcessor** edits definitions (placeholder `${}` resolution happens here).
- Instantiate (constructor / factory method).
- Populate properties / autowire.
- Aware callbacks (`BeanNameAware`, `ApplicationContextAware`, …).
- **BeanPostProcessor.postProcessBeforeInitialization**.
- `@PostConstruct` / `InitializingBean.afterPropertiesSet` / custom initMethod.
- **postProcessAfterInitialization** — AOP proxies (`@Transactional`, `@Async`, `@Secured`) are often wrapped here.
- Bean is ready. On shutdown (singletons): `@PreDestroy` / `DisposableBean`.

Scopes: **singleton** (default, one per container), **prototype** (new instance per lookup; container does not call destroy), request/session for web. Injecting a prototype into a singleton captures one instance forever unless you use `ObjectProvider`, `@Lookup`, or a scoped proxy.

5.3 REST APIs

- `@RestController` = `@Controller` + `@ResponseBody` (return value → HTTP body, typically JSON via Jackson).
- `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`.
- Idempotent: GET, PUT, DELETE (by spec intent). POST is not. PATCH may or may not be.
- PUT replaces the resource; missing fields often become null. PATCH is partial update — use a DTO with Optional fields or JsonNullable to distinguish 'absent' vs 'null'.
- Status codes: 201 + Location on create, 204 on empty delete, 400 validation, 401/403 security, 404 missing, 409 conflict, 429 rate limit.
- Versioning: URL `/api/v1`, header, or content-type. Pick one and be consistent.
- Validation: `@Valid` + `jakarta.validation` constraints on DTOs, not on entities if you can help it.

```
@RestController
@RequestMapping("/api/v1/orders")
@RequiredArgsConstructor
class OrderController {
    private final OrderService service;
    @PostMapping
```

```
    @ResponseStatus(HttpStatus.CREATED)
    OrderResponse create(@Valid @RequestBody CreateOrderRequest req) {
        return service.create(req);
    }
}
```

5.4 JPA / Hibernate

JPA is the specification (EntityManager, annotations). Hibernate is the most common provider. Spring Data JPA gives `JpaRepository` and query derivation.

- **Entity**: `@Entity`, `@Id`, identity vs sequence vs UUID. Equals/hashCode as discussed in Core Java.
- **Relationships**: `@ManyToOne` (owning side usually), `@OneToMany(mappedBy=...)`. Prefer lazy `FetchType.LAZY` (default for to-many).
- **N+1 problem**: one query for parents + one per parent for children. Fix: `join fetch` in JPQL, `@EntityGraph`, or batch size. Prove it with `show-sql` or p6spy, not vibes.
- **OSIV (Open Session In View)** is on by default in Boot and hides lazy-load errors by keeping the session open during JSON rendering. Many teams turn it off so service layer returns DTOs.
- **@Transactional**: proxy-based. Self-invocation does not start a transaction. Default rollback is RuntimeException, not checked exceptions.
- Propagation: REQUIRED (join or create), REQUIRES_NEW (suspend and start new), NESTED (savepoints). MANDATORY / NEVER / SUPPORTS / NOT_SUPPORTED exist — know REQUIRED and REQUIRES_NEW well.
- Dirty checking: loaded entities are snapshots; flush at commit. Do not use entities as API DTOs.
- Second-level cache vs query cache: know they exist; Redis is usually a **different** cache in front of services, not Hibernate L2 unless configured.

Interview trap: Never call `save()` and assume that is the only SQL. Updates often happen on flush because the instance is managed. Calling save on a managed entity is often a no-op besides returning the same instance.

5.5 Exception handling

```
@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    ProblemDetail notFound(NotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    Map<String, String> validation(MethodArgumentNotValidException ex) {
        // field -> message
        return Map.of();
    }
}
```

Prefer a consistent error payload (RFC 7807 Problem Detail is built into Spring 6). Log with a correlation id. Do not leak stack traces to clients. Translate persistence exceptions at the service/adapter boundary, not as raw SQLException JSON.

5.6 Spring Security

Security is a **filter chain** in front of the dispatcher servlet. Each request passes filters: SecurityContext persistence, logout, username-password or JWT, exception translation, authorization. The `SecurityFilterChain` bean is the modern replacement for `WebSecurityConfigurerAdapter` (removed).

- **Authentication**: who are you? (credentials → Authentication object in SecurityContext).
- **Authorization**: what can you do? (`authorizeHttpRequests`, method security `@PreAuthorize`).
- Password storage: DelegatingPasswordEncoder (bcrypt by default). Never log passwords.
- CSRF: required for cookie-session browser apps; typically disabled for stateless JWT APIs, but then you must not use cookie auth without other defenses.

• CORS is not security by itself; it is a browser policy. Still configure it explicitly for SPAs.

5.7 JWT and OAuth2

JWT (JSON Web Token): three Base64url parts — header, payload, signature (`alg` + secret/HMAC or private key/RSA/EC). Stateless: the resource server validates signature and `exp`/`iss`/`aud` without a session store. Drawbacks: hard to revoke before expiry (use short TTL + refresh tokens, or a denylist in Redis). Do not put secrets or huge PII in the payload; it is only encoded, not encrypted (unless you use JWE).

OAuth2: delegation protocol. Roles: Resource Owner (user), Client (your app), Authorization Server (issues tokens), Resource Server (API). Grant types: authorization code + PKCE (public SPAs/mobile), client credentials (machine-to-machine), refresh token. Implicit grant is obsolete. **OIDC** is identity on top of OAuth2 (id_token). Spring: `spring-boot-starter-oauth2-resource-server` for validating JWTs; `oauth2-client` for login. Do not write your own JWT parser crypto.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.csrf(csrf -> csrf.disable())
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
    return http.build();
}
```

5.8 Redis

Redis is an in-memory data structure server: strings, hashes, lists, sets, sorted sets, streams. In Spring Boot you typically use it for:

• **Cache**: `@EnableCaching` + `RedisCacheManager`. `@Cacheable`, `@CachePut`, `@CacheEvict`. Pick TTLs per cache. Cache stampsede: use locking or random jitter on TTL.

• **Session store**: Spring Session Redis for sticky-less web farms.

• **Rate limiting / distributed locks**: SET NX EX (or Redisson). Know that a naive lock is unsafe without fencing tokens.

• **Queues / pub-sub**: simple; Kafka is the better fit for durable event logs.

• Lettuce is the default driver (async, netty). Jedis is the older sync client.

Interview trap: Cache invalidation is the hard part. Cache **values** that are expensive and stable (product catalog), not rapidly mutating financial balances, unless you have a strict eviction story.

6. Microservices

A microservice is an independently deployable unit with its own data and a clear bounded context. You pay with network, ops, and data consistency. Interviewers want **trade-offs**, not 'Netflix stack' recitation.

6.1 API Gateway

A gateway is the single entry point for clients. Responsibilities: routing, TLS termination, authn/authz, rate limiting, request logging, protocol translation (REST → gRPC), aggregation (BFF). Spring Cloud Gateway is reactive (WebFlux) and route-based. Netflix Zuul 1 was servlet-based and is legacy. Keep **business logic out of the gateway**. If every feature needs a gateway change, you have built a monolith at the edge.

6.2 Service discovery

Instances come and go. Clients need a living address book.

- **Client-side discovery**: the caller queries a registry (Eureka, Consul, Kubernetes DNS/Endpoints) and picks an instance (load balancer in the client). Spring Cloud LoadBalancer + Eureka is the classic interview answer.
- **Server-side discovery**: the caller hits a VIP / kube Service / AWS ALB; the platform load-balances. In Kubernetes you often skip Eureka entirely.
- Health checks remove bad instances. Prefer `/actuator/health` with liveness vs readiness: liveness = restart me; readiness = do not send traffic (warm-up, DB down).

6.3 Communication

- **Synchronous REST/gRPC**: simple, strongly coupled in time. Use timeouts, retries with jitter, and idempotency keys on POSTs that may be retried.
- **Asynchronous messaging** (Kafka, RabbitMQ): temporal decoupling, backpressure, replay (Kafka). Harder end-to-end tracing and 'read-your-write' UX.
- **Saga**: long-running business transaction across services. Choreography (events) vs orchestration (a coordinator). Compensating actions, not 2PC (2PC does not scale across teams/DBs).
- **Outbox pattern**: write the DB row and an outbox event in one local transaction; a publisher relays to Kafka. Avoids 'saved DB but crashed before publish'.
- Contract testing (Spring Cloud Contract, Pact) beats hoping consumers read your wiki.

6.4 Circuit breaker

When a dependency is sick, failing fast is better than exhausting your own thread pool. The circuit is **Closed** (calls pass; failures counted) → **Open** (calls fail immediately or fallback) → **Half-open** (trial requests). Resilience4j is the current Spring Cloud default (Hystrix is in maintenance/retired). Combine: timeout + retry (idempotent only) + circuit breaker + bulkhead (isolate thread pools or concurrency limits) + fallback.

```
@CircuitBreaker(name = "payments", fallbackMethod = "payFallback")
@TimeLimiter(name = "payments")
public CompletableFuture<PayResult> pay(PayCmd cmd) { ... }

PayResult payFallback(PayCmd cmd, Throwable ex) {
    return PayResult.queuedForRetry(); // or fail with 503
}
```

Interview trap: A fallback that hits another overloaded service is not a fallback. Returning stale cache or 'accepted for later' is. Never retry non-idempotent POSTs blindly.

6.5 Tracing and observability

Logs, metrics, and traces answer different questions. **Logs**: what happened to this request. **Metrics**: how is the system now (RED: rate, errors, duration). **Traces**: where time went across services.

• Put a **correlation / trace id** on every log line (Micrometer Tracing, formerly Sleuth). Propagate `traceparent` (W3C) over HTTP and Kafka headers.

• Spring Boot Actuator + Micrometer + Prometheus + Grafana is the default metrics path. Traces go to Zipkin, Jaeger, or an OTLP backend.

• Sample traces in production (1% + always-on for errors) or you will drown in volume.

• SLOs (e.g. 99% of checkout < 300ms) beat 'the CPU looks fine'.

6.6 Load balancing

• **Client-side**: Spring Cloud LoadBalancer (round-robin, random, weighted; retry on next instance). Ribbon is legacy.

• **Server-side**: Kubernetes Service, AWS ALB/NLB, Nginx, API Gateway.

• **Sticky sessions**: avoid; store session in Redis or go stateless JWT.

• Health-aware LB: do not send traffic to instances failing readiness.

• gRPC and HTTP/2 need L7 balancers that understand the protocol; a naive L4 round-robin can pin long-lived connections.

Remember: If the environment is Kubernetes, say: 'Service discovery and load balancing are kube Services + Ingress/Gateway; Spring Cloud Netflix is optional, not mandatory.'

7. CI/CD and Cloud

7.1 Git

- `git add` (stage) → `commit` (local snapshot) → `push` (remote). `pull` = fetch + merge (or rebase if you configured it).
- Branching: trunk-based with short-lived feature branches is what most CI-friendly teams use. GitFlow is heavier (develop/release/hotfix).
- `merge` creates a merge commit; `rebase` replays your commits on top of latest main (cleaner history, rewriting published history is rude).
- PR hygiene: small diffs, green pipeline, meaningful description. Never commit secrets. `.gitignore` for `target/`, `.idea/`, `.env`.
- `git revert` is the safe undo on shared branches; `reset --hard` is local only unless you force-push (usually forbidden on main).

7.2 Maven

Maven is a build + dependency tool. The **lifecycle** is a chain of phases: `validate` → `compile` → `test` → `package` → `verify` → `install` (local `.m2`) → `deploy` (remote repo). A plugin goal binds to a phase. `pom.xml` coordinates: groupId:artifactId:version. Dependency scopes: `compile` (default), `provided` (servlet API), `runtime` (JDBC driver sometimes), `test`, `import` (BOMs). A **BOM** (`spring-boot-dependencies`) aligns versions. `mvn -am -pl module` builds a module and its dependents. The `target/` directory is output; do not commit it.

Interview trap: Gradle vs Maven is a tooling preference. If the job uses Maven, speak Maven fluently: SNAPSHOT vs release, and why SNAPSHOTS should not ship to prod.

7.3 Jenkins and GitLab CI

CI: every push runs compile, unit tests, static analysis. CD: deploy artifacts automatically to an environment with gates.

Jenkins

- Declarative `Jenkinsfile` in the repo (pipeline as code): stages for build, test, image, deploy.
- Agents/executors run jobs; use containers as agents for clean Maven/JDK images.
- Credentials stored in Jenkins credentials store, injected as env vars — never hardcoded.
- Shared libraries for repeated pipelines. Webhooks from Git to trigger.

GitLab CI

- `.gitlab-ci.yml`: jobs, stages, `image: maven:3.9-eclipse-temurin-21`, artifacts, cache (`~/.m2`).
- `rules` / `only: merge_requests` to control when jobs run.
- Environments and manual `when: manual` jobs for prod. Protected branches + protected variables.
- GitLab Container Registry often holds the Docker image the deploy job pulls.

```
# .gitlab-ci.yml (sketch)
stages: [build, test, image, deploy]
build:
  stage: build
  script: mvn -B -DskipTests package
  artifacts: { paths: [target/*.jar] }
test:
  stage: test
  script: mvn -B test
image:
  stage: image
  script: docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
deploy_prod:
  stage: deploy
  when: manual
  script: helm upgrade --install api charts/api
```

7.4 Docker

A **image** is a layered immutable filesystem + metadata. A **container** is a running instance (namespaces + cgroups). Dockerfile: FROM a JDK or JRE base, COPY the jar, USER non-root, EXPOSE, ENTRYPOINT. Prefer a small JRE image or distroless. Multi-stage builds: compile in a JDK image, copy the jar into a slim runtime image so compilers never ship to prod.

```
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /src
COPY pom.xml .
COPY src ./src
RUN mvn -q -DskipTests package

FROM eclipse-temurin:21-jre-alpine
RUN adduser -S app
USER app
COPY --from=build /src/target/app.jar /app.jar
ENTRYPOINT ["java","-XX:+UseContainerSupport","-jar","/app.jar"]
```

- 12-factor: config via env vars (`SPRING_DATASOURCE_URL`), not baked into the image.
- `docker-compose` for local Kafka/Redis/Postgres (as in a kafka-demo style project).
- Kubernetes (orchestration) is not Docker, but it runs containers: Pods, Deployments, Services, ConfigMaps, Secrets, HPA.

7.5 Deployment

- Artifact: versioned jar or image tag (`git sha`), never `latest` in production.
- Rolling deploy: new pods come up, readiness gate, old pods drain. Blue/green: two full environments, flip the router. Canary: send 5% traffic, watch error rate, then promote.
- Migrations: Flyway/Liquibase **expand/contract** (additive first) so old and new app versions can run together during a roll-out.
- Rollback: redeploy previous image; DB rollbacks are harder — design forward-compatible schemas.
- Health, metrics, and logs must exist before the first prod deploy, not after the first outage.

7.6 AWS basics

You are not expected to be a solutions architect. You are expected to place a Spring Boot service on AWS without hand-waving.

- **IAM**: users, roles, policies. EC2/ECS tasks assume a **role**. Access keys in code are a finding, not a design. Least privilege.
- **VPC**: private subnets for app + RDS, public subnet for load balancer. Security groups as stateful firewalls.
- **EC2**: VMs. You manage OS and the jar. Simple, operationally heavy.
- **ECS/Fargate** or **EKS**: run containers. Fargate = no node management. EKS = Kubernetes.
- **Elastic Beanstalk**: PaaS-ish for jars; less common in newer orgs than ECS.
- **ALB**: L7 HTTP load balancer, path routing, TLS certs (ACM).
- **RDS**: managed Postgres/MySQL. Multi-AZ for HA. Parameter groups, backups, not a substitute for app-level pooling (HikariCP).
- **S3**: object storage for files, reports, static assets. Pre-signed URLs instead of public buckets.
- **SQS / SNS**: queues and pub-sub. Kafka on MSK if you need a log. Do not confuse SQS (pull queue) with SNS (fan-out).
- **ElastiCache**: managed Redis/Memcached.
- **CloudWatch**: logs and metrics. **X-Ray** or OpenTelemetry for traces.
- **Secrets Manager / SSM Parameter Store**: DB passwords, JWT secrets, rotated.
- **CloudFront**: CDN in front of S3 or ALB.

A standard interview architecture: Route 53 → ALB → ECS services (Spring Boot) in private subnets → RDS + ElastiCache Redis; images in ECR; pipeline in GitLab/Jenkins/CodePipeline; logs to CloudWatch; config from SSM; each service an IAM task role. That paragraph is enough if you can also say **why** the DB is not public and **why** the JWT secret is not in Git.

Remember: When they ask 'Docker vs VM': isolation and density vs full OS. When they ask 'Docker vs Kubernetes': Kubernetes schedules, heals, and scales **containers**; Docker builds and runs one of them.

Quick verbal checklists

90-second drills

- HashMap vs ConcurrentHashMap: bins, treeify 8, CAS empty bin, per-bin lock, no nulls in CHM.
- equals/hashCode: equal objects → equal hashes; no mutable keys; JPA id pitfall.
- String pool: literals interned; new String() is a new object; immutability enables sharing.
- volatile vs synchronized: visibility vs mutual exclusion.
- SOLID: one sentence each plus a Spring example for D and O.
- Singleton: enum or holder; Spring singleton ≠ JVM singleton.
- map vs flatMap: one-to-one wrap vs one-to-many flatten.
- Optional: return type only; orElse vs orElseGet.
- @Transactional: proxy, no self-invocation, rollback on unchecked only by default.
- JWT vs OAuth2: token format vs delegation protocol; resource server validates JWT.
- Circuit breaker: fail fast + fallback; Resilience4j; do not retry POSTs blindly.
- Maven: compile-test-package-install-deploy.
- AWS sketch: ALB → ECS → RDS, secrets not in Git, private subnets.

If you can teach any row above to another engineer without looking at this PDF, you are ready for that slice of the interview.